5.7 Transformer

- we have compared CNNs, RNNs, and self-attention in Section 5.6
- notably, self-attention enjoys both parallel computation and the shortest maximum path length
- therefore, natually, it is appealing to design deep architectures by using self-attention
- unlike earlier self-attention models that still rely on RNNs for input representations, the *transformer* model is solely based on attention mechanisms, without any convolutional or recurrent layer

5.7 Transformer

- though originally proposed for sequence to sequence learning on text data, transformers have been pervasive in a wide range of modern deep learning applications, such as in areas of language, vision, speech, and reinforcement learning
- as an instance of the encoder-decoder architecture, the overall architecture of the transformer is presented in Figure 9
- as we can see, the transformer is composed of an encoder and a decoder
- different from Bahdanau attention for sequence to sequence learning in Figure 5, the input (source) and output (target) sequence embeddings are added with positional encoding, before being fed into the encoder and the decoder that stack modules based on self-attention

5.7 Transformer

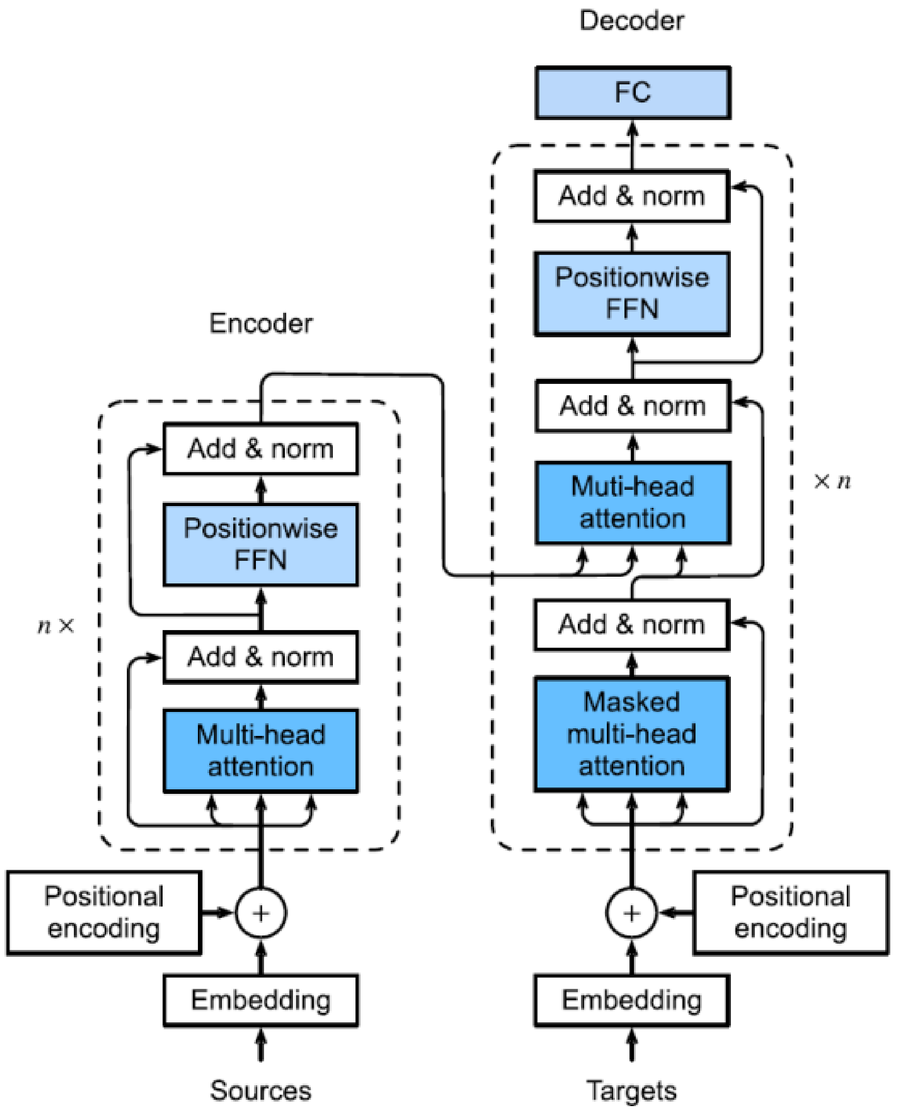


Figure 9: The transformer architecture.

5.7 Transformer

- now, we provide an overview of the transformer architecture in Figure 9
- on a high level, the transformer encoder is a stack of multiple identical layers, where each layer has two sublayers (either is denoted as sublayer)
- the first is a multi-head self-attention pooling and the second is a position-wise feed-forward network
- specifically, in the encoder self-attention, queries, keys, and values are all from the outputs of the previous encoder layer

5.7 Transformer

- inspired by the ResNet design in Section 3.10, a residual connection is employed around both sublayers
- in the transformer, for any input $\boldsymbol{x} \in \mathbb{R}^d$, at any position of the sequence, we require that sublayer$(\boldsymbol{x}) \in \mathbb{R}^d$, so that the residual connection $\boldsymbol{x} + \text{sublayer}(\boldsymbol{x}) \in \mathbb{R}^d$ is feasible
- this addition from the residual connection is immediately followed by *layer normalization*
- as a result, the transformer encoder outputs a d-dimensional vector representation for each position of the input sequence

5.7 Transformer

- the transformer decoder is also a stack of multiple identical layers with residual connections and layer normalizations
- besides the two sublayers described in the encoder, the decoder inserts a third sublayer, known as the *encoder-decoder attention*, between these two
- in the encoder-decoder attention, queries are from the outputs of the previous decoder layer, and the keys and values are from the transformer encoder outputs
- in the decoder self-attention, queries, keys, and values are all from the outputs of the previous decoder layer
- however, each position in the decoder is allowed to only attend to all positions in the decoder up to that position
- this *masked* attention preserves the auto-regressive property, ensuring that the prediction only depends on those output tokens that have been generated

5.7 Transformer

- the position-wise feed-forward network transforms the representation at all the sequence positions using the same MLP; this is why we call it *position-wise*
- now, let us focus on the “add & norm” component in Figure 9
- as we described at the beginning of this section, this is a residual connection immediately followed by layer normalization; both are key to effective deep architectures
- in Section 3.9, we explained how batch normalization recenters and rescales across the examples within a mini-batch
- layer normalization is the same as batch normalization, except that the former normalizes across the feature dimension
- despite its pervasive applications in computer vision, batch normalization is usually empirically less effective than layer normalization in natural language processing tasks, whose inputs are often variable-length sequences

5.7 Transformer

- when training sequence-to-sequence models, tokens at all the positions (time steps) of the output sequence are known
- however, during prediction, the output sequence is generated token by token; thus, at any decoder time step, only the generated tokens can be used in the decoder self-attention
- to preserve auto-regression in the decoder, its masked self-attention specifies that any query should only attend to all positions in the decoder up to the query position
- although the transformer architecture was originally proposed for sequence-to-sequence learning, either the transformer encoder or the transformer decoder is often individually used for different deep learning tasks